ХАКАТОН

AI-агент для разработки ML-моделей

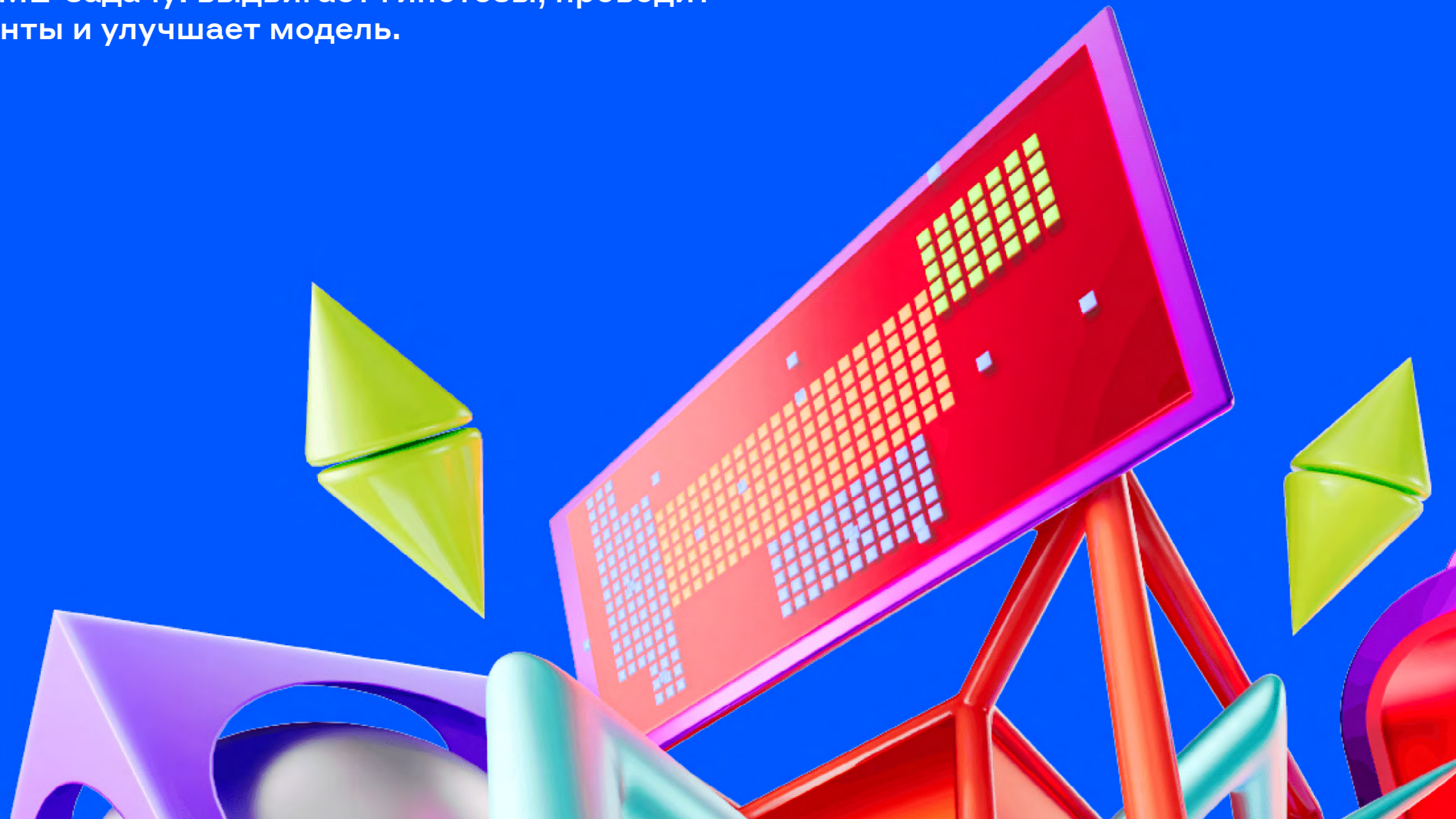


АЛЬФА БУДУЩЕЕ

Альфа Банк

Аннотация

Что, если ML-инженер сможет поручить AI-агенту не только обучение модели, но и сам поиск решения? Вам предстоит создать агента, который самостоятельно исследует ML-задачу: выдвигает гипотезы, проводит эксперименты и улучшает модель.



Оглавление

Бизнес-контекст 04

Постановка задачи 05

Ограничения 07

Пространство решений 08

Описание данных 09

Как устроен процесс создания ML-модели 11

Что такое агентная система и чем она отличается от обычного AutoML 13

Цикл AutoResearch 14

Приложения 16

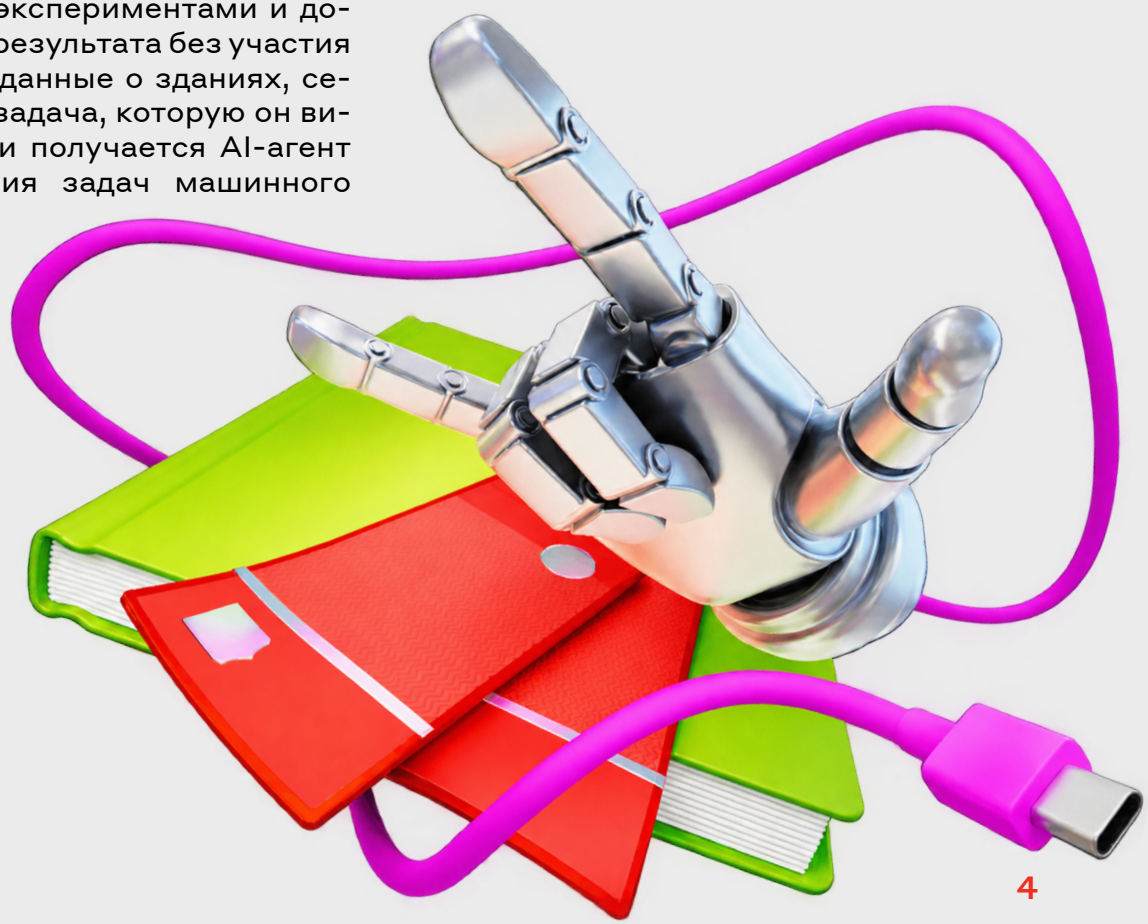
Бизнес-контекст

Машинное обучение давно стало рабочим инструментом бизнеса: на моделях строятся прогноз спроса, скоринг, поиск мошенничества, рекомендации. Число таких задач в компаниях растет быстрее, чем число ML-инженеров.

Узкое место — не в обучении модели, а в поиске решения. Для каждой новой задачи специалист проходит один и тот же исследовательский путь: разбирается в данных, проверяет несколько подходов, сравнивает результаты, возвращается к признакам и валидации. Этот цикл занимает дни и почти не переносится между задачами. Масштабировать его сегодня можно только одним способом — нанимать больше людей.

AutoML снимает часть рутины: перебирает алгоритмы и гиперпараметры внутри заранее заданного сценария. Но он не задается вопросом: почему модель ошибается и что стоит попробовать иначе?

Эту роль может взять на себя AI-агент: изучить новую задачу, выдвинуть гипотезы, проверить их экспериментами и довести решение до результата без участия человека, будь то данные о зданиях, сетевой трафик или задача, которую он видит впервые. Так и получается AI-агент end-to-end-решения задач машинного обучения.



Постановка задачи

Создайте AI-агента, который самостоятельно решает задачи машинного обучения — от чтения условия до готового файла с предсказаниями.
Для этого:

1

Научите агента анализировать текстовое описание задачи, изучать структуру входных данных, определять тип ML-задачи и требования к результату.

2

Автоматизируйте построение решения, где агент сам выбирает подход, генерирует и запускает код, обучает модель, оценивает качество на собственной валидации и формирует итоговые предсказания.

3

Добавьте цикл AutoResearch. Агент анализирует результаты экспериментов, формулирует гипотезы для улучшения модели, проверяет их и выбирает лучший вариант решения.

4

Обеспечьте универсальность. Один и тот же код должен решать разные задачи, включая классификацию и регрессию и другие задачи машинного обучения, без ручной настройки под конкретный датасет.

5

Отправьте агента на платформу. Решение проверяется на двух публичных задачах, а итоговый результат определяется на двух скрытых, которые агент увидит впервые (подробнее — в приложении 1).

Постановка задачи

ФОРМАТ СДАЧИ

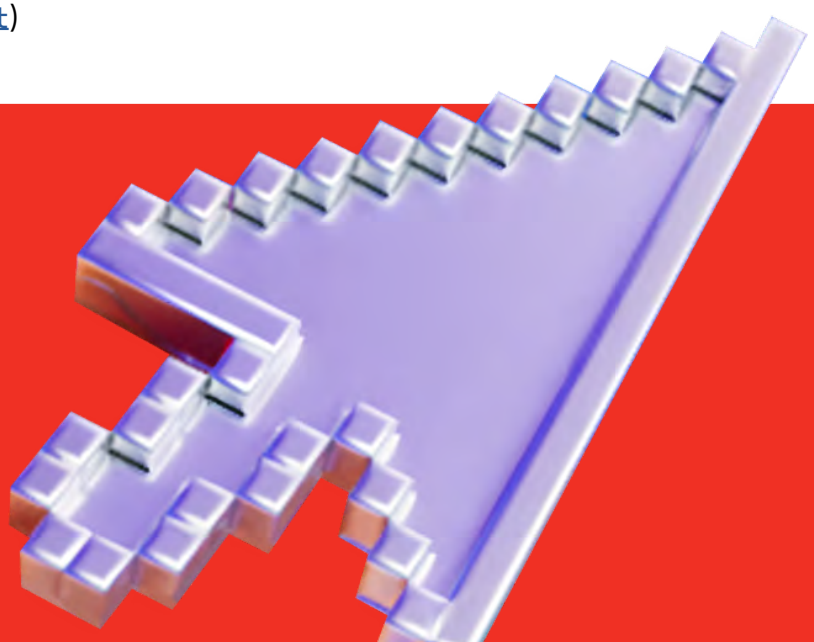
ссылка на baseline

На платформу загружается ZIP-архив с решением. В корне архива должен лежать файл solution.py — единая точка входа для запуска агента. Логику можно распределить по нескольким модулям внутри того же архива: solution.py импортирует их и вызывает нужный код. Платформа запускает решение в Docker-контейнере.

Переменные окружения, интерфейс запуска и формат вывода зафиксированы в baseline.py. Решение должно строго соблюдать этот интерфейс, иначе платформа не сможет его проверить.

Важно! Число сабмитов в день на команду ограничивается 3. Количество сабмитов обновляется в 00:00 мск каждый день.

Доступные библиотеки (ссылка на requirements.txt)



Ограничения

01

Для создания и настройки модели вы можете использовать язык Python версии 3.12.

02

Проверка решений проводится на платформе в Docker-контейнере на единых вычислительных мощностях: 8 CPU, 32 GB RAM.

03

На отборочном этапе участники используют собственные мощности для разработки, всем участникам предоставляется доступ через API к генеративной модели Qwen3.8-27B. Использовать другие модели через собственные API-ключи запрещено.

04

На время разработки команда получает общий ключ доступа к языковой модели с дневным лимитом на количество токенов, ограничение в денежном эквиваленте — 600 рублей.

05

На одну задачу у агента на платформе есть не более 3 часов для решения задачи. После этого процесс останавливается и оценивается уже сформированный файл предсказаний. Если за 3 часа работы скрипт не вернул результат, запуск прекращается.

06

Для обучения используются только данные, предоставленные в кейсе. Предобученные модели использовать нельзя. Доступ в интернет из контейнера запрещен, для обучения вы можете использовать только модели, зафиксированные в requirements.txt. Если вы хотите использовать в решении модели, не входящие в этот список, свяжитесь с организаторами в чате хакатона.

07

Формирование предсказаний по задаче должно формироваться в формате, зафиксированном в baseline.py, финальный скор решения рассчитывается по формуле, представленной в приложении к кейсу (приложение 3).

Пространство решений

1

Допускается одноагентная или мультиагентная система. Команда сама определяет роли агентов и способ их взаимодействия.

2

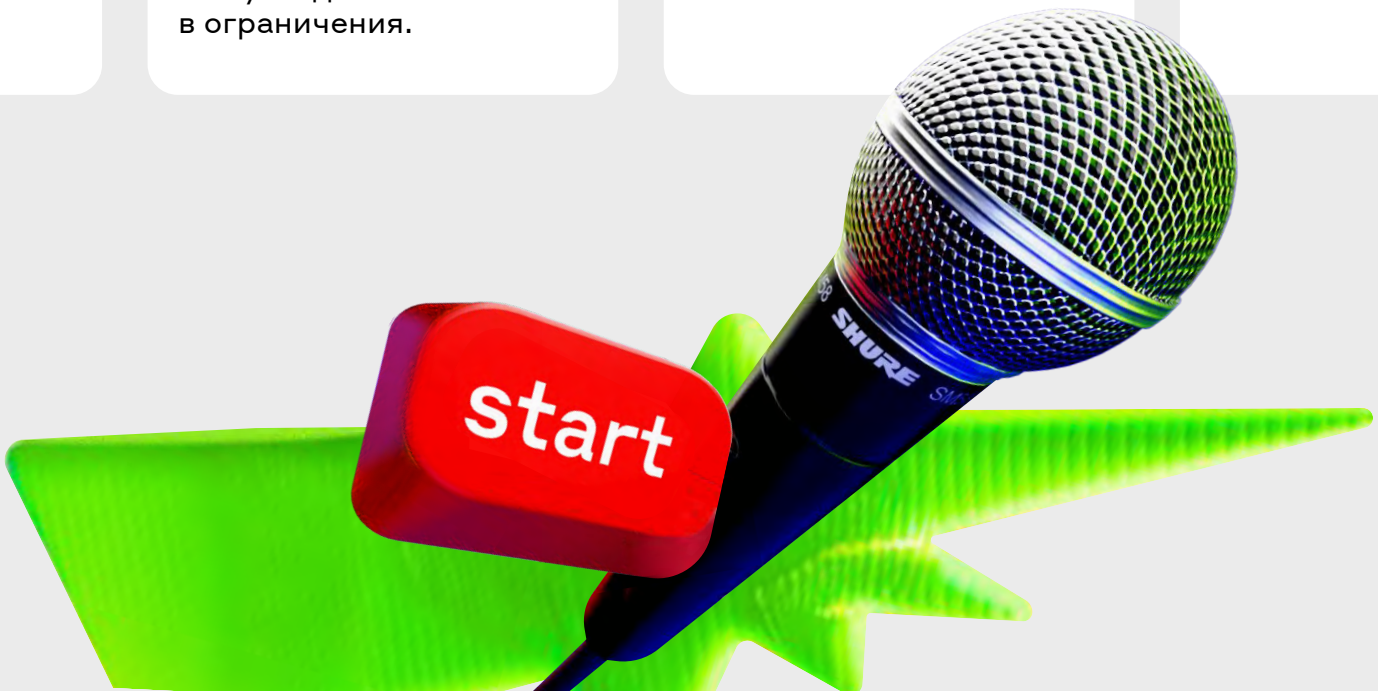
Агент может использовать любые алгоритмы машинного обучения (бустинги, нейросети, линейные модели, ансамбли), библиотеки, схемы валидации и методы подбора гиперпараметров, если они укладываются в ограничения.

3

В цикле AutoResearch агент может проверять гипотезы о выборе алгоритма, гиперпараметрах, предобработке и очистке данных, отборе и генерации признаков, схеме валидации.

4

Команда сама решает, как распределить 3 часа и лимит обращений к LLM между исследованием и финальным обучением: например, быстро получить надежный базовый результат, а оставшееся время тратить на улучшения.



Описание данных

ссылка на данные

Все CSV-файлы используют кодировку UTF-8, запятую как разделитель и точку как десятичный разделитель. Пустая ячейка означает пропуск.

В каждой задаче агент получает одинаковый набор файлов: train.csv, test.csv, data_dictionary.csv и sample_submission.csv.

Задача 1. Определение годового удельного энергопотребления зданий (регрессия).

train.csv	Обучающая выборка состоит из 64 столбцов: • 62 входных признака; • идентификатор ‘id‘ и таргет ‘site_eui‘. Объем — 53 308 строк
test.csv	Тестовая выборка состоит из 63 столбцов: те же признаки и `id`, без таргета. Объем — 22 449 строк
data_dictionary.csv	Описание столбцов обучающей и тестовой выборок, состоит из 5 столбцов: • название признака ‘feature’; • роль признака в датасете ‘role’; • тип данных ‘dtype’; • формат ‘unit’; • описание признака на русском языке ’description_ru’. Описывает все 64 столбца train.csv
sample_submission.csv	Пример сабмита состоит из 2 колонок: ‘id’ и ‘site_eui’

Метрика: RMSE.

Описание данных

Задача 2. Определение типа кибер-атаки (многоклассовая классификация).

train.csv	Обучающая выборка состоит из 44 столбцов: • 42 входных признака; • идентификатор `id` и таргет ‘attack_cat‘. Объем — 87 670 строк
test.csv	Тестовая выборка состоит 43 столбцов: те же признаки и `id` без таргета. Объем — 41 166 строк
data_dictionary.csv	Описание столбцов обучающей и тестовой выборок, состоит из 5 столбцов: • название признака ‘feature’; • роль признака в датасете ‘role’; • тип данных ‘dtype’; • описание признака на русском языке ’description_ru’. Описывает все 44 столбца train.csv
sample_submission.csv	Пример сабмита состоит из 2 колонок: ‘id’ и ‘attack_cat’

Основная метрика: Macro F1 (учитывается в итоговом score). Accuracy и Weighted F1 используются как справочные.

Как устроен процесс создания ML-модели

Разработка ML-модели — итеративный процесс. Специалист раз за разом возвращается к данным, признакам и настройкам, пока не получит нужное качество. На каждом этапе он принимает решения, опираясь на опыт и интуицию. В этом кейсе все эти решения предстоит принимать агенту.

Этап	Что делает специалист	Какое решение придется принять агенту
Понимание задачи	Читает условие, разбирается в предметной области и метрике	Какой это тип задачи и что именно нужно оптимизировать?
Анализ данных	Изучает структуру датасета, типы признаков, пропуски, выбросы, распределение целевой переменной	Какие особенности данных важны для решения?
Предобработка	Очищает данные, обрабатывает пропуски, кодирует категориальные признаки	Как подготовить данные, не зная их заранее?
Формирование признаков	Создает новые признаки и отбирает полезные	Какие признаки стоит попробовать и как понять, что они помогли?

Как устроен процесс создания ML-модели

Этап	Что делает специалист	Какое решение придется принять агенту
Моделирование	Выбирает тип модели и настраивает гиперпараметры	С какой модели начать и когда переключаться на другую?
Валидация	Проверяет качество на отложенной выборке или кросс-валидации	Как оценивать гипотезы так, чтобы результат держался и на скрытом тесте?
Итерация	Анализирует ошибки и решает, куда двигаться дальше	Что проверить следующим и когда остановиться?

В промышленной практике к этому циклу добавляются развертывание модели в production и мониторинг ее качества на новых данных, а весь процесс сопровождается подходами MLOps: версионированием кода и данных, управлением экспериментами, воспроизводимостью. В этом кейсе агенту не нужно развертывать модель, но идея управления экспериментами остается важной: чтобы выбрать лучшее решение, агенту нужно помнить, что он уже пробовал и с каким результатом.

Именно сложность этого цикла и то, что каждое решение в нем требует экспертизы, стали одним из главных факторов развития AutoML и AI-агентов.

Что такое агентная система и чем она отличается от обычного AutoML

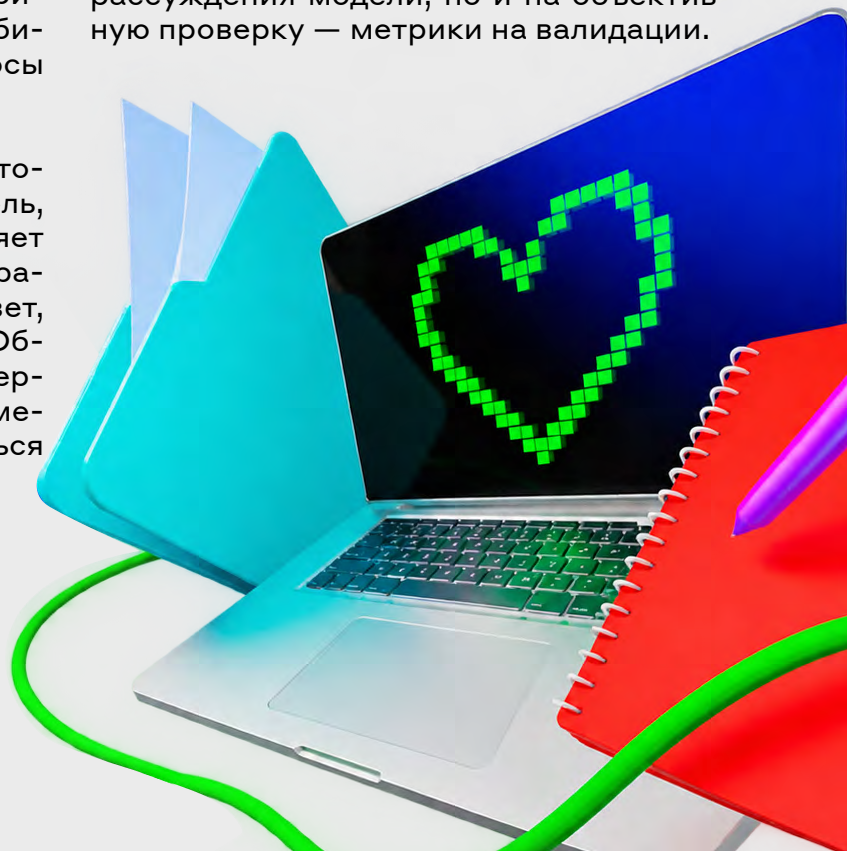
AutoML автоматизирует отдельные этапы ML-пайплайна. Например, сервисы вроде Azure AutoML параллельно перебирают алгоритмы и гиперпараметры, строят множество моделей и выбирают лучшую по заданной метрике. Это ускоряет работу, но AutoML действует в рамках заранее заданного сценария: он не анализирует, почему модель ошибается, и не придумывает подходов, которые не заложили его разработчики.

Агентная система устроена иначе. Она состоит из одного или нескольких агентов под управлением LLM, каждый со своей ролью: например, один анализирует данные, другой предлагает гипотезы, третий оценивает результаты. Агент может планировать последовательность действий, писать и запускать код, комбинировать инструменты и менять стратегию по ходу работы.

Такой подход уже используется в разных областях. Claude Code — кодинг-агент, который сам планирует работу, пишет и запускает код, анализирует результаты и исправляет ошибки. «Нейроюрист» от «Яндекса» — AI-помощник для юристов на базе YandexGPT, который подбирает правовую базу, отвечает на вопросы и анализирует документы.

По описанию его разработчиков, на стороне сервиса работает не одна модель, а система из нескольких: одна уточняет запрос, другая определяет область права и ищет данные, третья готовит ответ, четвертая перепроверяет выводы. Общее у таких систем то, что агент удерживает контекст задачи, помнит промежуточные результаты и может вернуться к предыдущим шагам.

У агентов есть и ограничения, во многом унаследованные от LLM: галлюцинации, сложность интерпретации решений и затраты на вычисления. Поэтому хорошая агентная система опирается не только на рассуждения модели, но и на объективную проверку — метрики на валидации.



Цикл AutoResearch

Любое исследование в машинном обучении устроено как цикл. Исследователь формулирует гипотезу, проверяет ее экспериментом, смотрит на результат и решает, что делать дальше. Подход AutoResearch переносит этот цикл в агентную систему:

01

Гипотеза.

Агент формулирует предположение, которое может улучшить результат. Например: «Добавление признака „возраст здания“ снизит ошибку прогноза».

02

Эксперимент.

Агент пишет и запускает код для проверки: генерирует признак, переобучает модель.

03

Оценка.

Результат сравнивается с предыдущим лучшим по метрике на валидационной выборке.

04

Следующая итерация.

Если гипотеза подтвердилась, изменение сохраняется. Если нет, агент анализирует причину и выбирает следующее направление.

При понятной цели и метрике каждая такая итерация во многом механическая, и главное ограничение человека здесь — скорость. Хороший исследователь успевает пройти 8–10 циклов за рабочий день, причем большую часть времени он ждет, пока обучится модель. Агент же работает без перерывов и не теряет фокус даже после десятой неудачной попытки подряд.

Цикл AutoResearch

Насколько это эффективно, хорошо показал проект autoresearch (о нем — в следующем разделе). Агента оставили на двое суток улучшать модель, которую считали уже хорошо оптимизированной. За это время агент внес около 700 изменений и нашел порядка 20 улучшений, которые перенеслись и на более крупные модели, ускорив обучение примерно на 11%. Среди находок были недочеты в масштабировании механизма внимания (attention) и регуляризации, которые до этого пропустили.

Вместе с этим меняется и роль человека. Исследователь все меньше пишет код обучения и все больше формулирует направления исследования: в autoresearch человек вообще не трогает Python-файлы, а пишет инструкцию на обычном языке о том, что исследовать и какие ограничения соблюдать. В этом кейсе в похожей роли оказываются участники: они проектируют агента, а задачи он решает уже без них.

У такой автономности есть обратная сторона. Агент оптимизирует ровно ту метрику, которую видит, и может научиться обманывать оценку вместо того, чтобы честно улучшать решение. Например, в собственном отчете Weco AI о системе, которая автоматически улучшала другого исследовательского агента, на одном из бенчмарков агенты взламывали метрику примерно в 63% случаев, пока система сама не выработала защиту и не снизила эту долю до 34%. Поэтому ключевым шагом цикла остается оценка: только честная проверка гипотез позволяет агенту действительно улучшать решение, а не подгоняться под метрику.



Приложение 1.
Публичная и приватная проверка

Public

- Две задачи — регрессия и многоклассовая классификация.
- Платформа рассчитывает метрики по скрытым целевым значениям тестовых выборок и объединяет результаты в publ

ic score.

- Участники видят результат на публичном лидерборде и могут отправлять новые версии решения в пределах лимита отправок.

Private

- Две новые задачи — содержание и метрики задач не раскрываются.
- Платформа выбирает сохраненную последнюю версию решения с лучшим public score и повторно запускает ее через solution.py.
- Вместо public-материалов агент получает описания и данные private-задач через тот же интерфейс, зафиксированный в baseline.py. Код решения остается неизменным.
- Платформа рассчитывает метрики и итоговый private score.
- Ограничения по времени и ресурсам, требования к формату вывода и принцип объединения результатов сохраняются.



Приложение 2.
Запуск и самостоятельная работа агента

Каждая отправка — это один сабмит. Для каждой задачи агент запускается отдельно: он получает материалы только этой задачи и формирует ответ для нее. Файлы и результаты, полученные при решении одной задачи, в следующую не передаются.

Внутри запуска агент работает самостоятельно: анализирует данные, организует валидацию на основе обучающей выборки, генерирует и исполняет код, обучает модели и проверяет гипотезы. Для сравнения экспериментов он сам рассчитывает метрики на своей валидационной выборке. Платформа не оценивает промежуточные предсказания на скрытом тесте и не возвращает агенту обратную связь.

На одну задачу отводится не более 3 часов, отсчет идет от запуска script.py. Агент может завершить работу раньше. По истечении 3 часов процесс принудительно останавливается, и к этому моменту файл предсказаний уже должен быть сформирован.



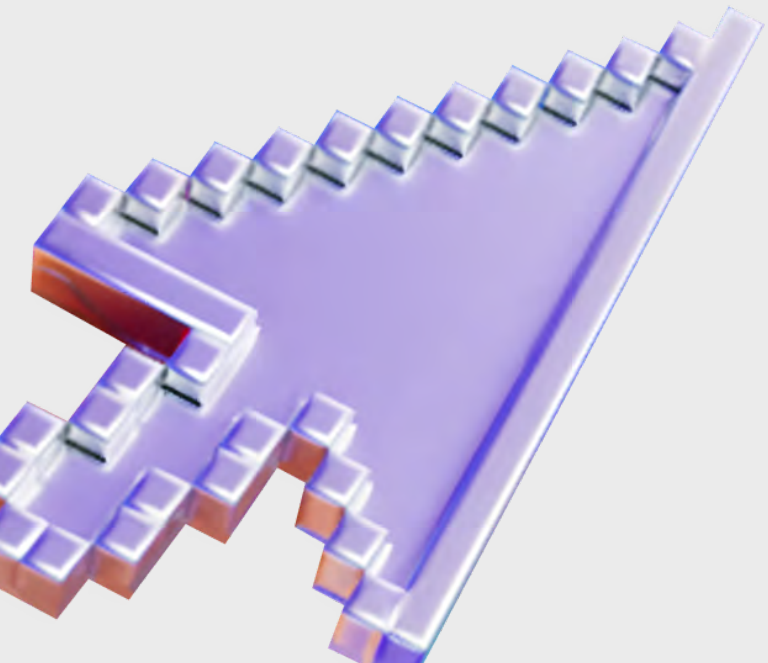
Приложение 3.
Расчет финальной метрики решения

На публичном этапе решение оценивается на двух задачах: регрессии и многоклассовой классификации.

1. Регрессия — RMSE

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2},$$

где n — число объектов тестовой выборки, y_i — истинное значение, $\hat{y}_i$ — предсказание агента. Чем меньше RMSE, тем лучше результат.



2. Многоклассовая классификация — Macro F1

Для каждого класса рассчитывается F1, затем результаты усредняются с одинаковым весом:

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k},$$

$$MacroF1 = \frac{1}{K}\sum_{k=1}^{K} F1_k$$

где K — число классов; TP_k, FP_k и FN_k — число верных положительных, ложных положительных и ложных отрицательных предсказаний для класса k. Чем выше Macro F1, тем лучше результат.

3. Перевод метрик в баллы

Метрика каждой задачи нормируется относительно результата бейзлайна на той же тестовой выборке:

$$S_{reg} = 100 \cdot \max\left(0, 1 - \frac{RMSE}{B_{RMSE}}\right)$$

$$S_{cls} = 100 \cdot \max\left(0, \frac{MacroF1 - B_{F1}}{1 - B_{F1}}\right),$$

где $B_{RMSE}>0$ и $B_{F1}<1$ — значения метрик бейзлайна. Результат на уровне бейзлайна или хуже дает 0 баллов, идеальный результат — 100 баллов.

Приложение 3.
Расчет финальной метрики решения

4. Итоговый score публичного лидерборда

Обе задачи имеют одинаковый вес:

$$Score_{public} = \frac{S_{reg}+S_{cls}}{2}$$

Итоговый score находится в диапазоне от 0 до 100. Чем выше score, тем выше позиция команды в лидерборде.

5. Итоговый score приватного лидерборда

Итоговый score приватного лидерборда рассчитывается как среднее арифметическое баллов по двум приватным задачам:

$$Score_{private} = \frac{S_1+S_2}{2},$$

где S_1 и S_2 — баллы по первой и второй приватным задачам соответственно.

Метрики задач могут различаться. Балл каждой задачи рассчитывается по формуле, соответствующей ее метрике.



CHANGELLENGE >>

Кейс написан и опубликован Changellenge >> — ведущей организацией по кейсам в России.

www.changellenge.com

АЛЬФА БУДУЩЕЕ

Альфа Банк

Кейс создан совместно с АО «Альфа-Банк»

www.alfabank.ru

